



MTU

Cliona Mc Guane

You will create the **core functionality** for this project as part of your coursework.

In **Week 11**, you will be required to **extend your program** by adding three additional features **under exam conditions**, without access to the internet. It is therefore essential that you fully understand the code you submit and how your program works.

You are required to use version control throughout the development of this project. Your repository should show **clear evidence of incremental development** over time, rather than a single large commit.

You should make **regular commits** as you build your solution, for example

-  after completing the file reading functionality
-  after implementing each menu option
-  after testing or fixing bugs.

Each **commit message** should clearly **describe what was added** or changed (e.g. "Added delete function", "Fixed bug in status update").

Your version control history will form part of the assessment, with **marks awarded for consistent and regular commits**, clear and meaningful commit messages, and evidence of step-by-step development.

Submitting a repository with only one or very few large commits will result in reduced marks, as it does not demonstrate your development process.

While you may choose to use tools such as AI to support your learning during development, **you must ensure that you document this in comments** and **understand any code you include in your project**. Your commit history should reflect your own understanding and progression, and **you may be asked to explain your code** and development decisions.

Marking Scheme

	Mark
VC (Git) on core functionality	16%
Option 5	28%
Option 6	28%
Option 7	28%

No marks are awarded for the core functionality but completing these tasks will contribute to the VC (Git) grade and will prepare you for the task of extending the functionality.

GitHub Account

[This tutorial was created with ChatGPT's help]

Before starting this project, create a Git repository on GitHub which I will use to assess the version control element of this project (16%).



1. Create a GitHub Account (with your MTU Email)

1. Go to GitHub → <https://github.com>
2. Click **Sign up for GitHub** using your MTU email address
3. Enter:
 - Your **MTU email address** (e.g. student@mtu.ie)
 - A secure password
 - A username (this will be public so make it professional)
4. Verify your email (check your MTU inbox)



Optional (but recommended)

- Go to **Settings** → **Emails**
 - Ensure your MTU email is **verified and primary**
-



2. Create a New Repository

1. Click the “+” (**top-right corner**) → **New repository**
 2. Fill in:
 - **Repository name** (e.g. programming-project, maths-assignment, etc.)
 - **Description** (optional)
 - Choose **Private (recommended)** for coursework) or Public
 3. Tick:
 - ☒ “Add a README file”
 4. Click **Create repository**
 5. Go to your repository page on GitHub
 6. Look for the green “**Code**” button (near the top-right of the file list)
 7. Click it
 8. You’ll see a URL box with options like:
 - **HTTPS** (most common)
 - SSH
 9. Copy the URL (it will look like this):
 - <https://github.com/your-username/your-repo-name.git>
-



3. Connect PyCharm to GitHub

1. Open PyCharm
 2. Go to
 - a. **Settings** → **Version Control** → **GitHub** → + sign → **Log in Via GitHub**
 3. Provide your credentials
-

4. Link Your Repository to PyCharm

1. In PyCharm: **File** → **New** → **Project from Version Control**
 2. Paste your repo URL (from GitHub page Step 2 above)
 3. Choose a folder → Clone
 4. It should download all files from your repository, in this case just README.md file but it has also made the connection between PyCharm's project and your GitHub repository.
-

5. Add & Commit Files

1. Add college.txt and starter code to your project in PyCharm
 2. **Commit and Push** these files to GitHub
 3. Take a look at your repository in GitHub through your browser to verify it worked.
 4. Create or add files in your project
-

6. Invite Your Lecturer to Your Repository

To allow your lecturer to view your work on GitHub:

1. Go to your repository page
 2. Click **Settings** (top menu of the repo)
 3. In the left sidebar, click **Collaborators**
 4. Click **Add people**
 5. Enter the email address: cliona.mcguane@mtu.ie
 6. Click **Add collaborator**
 7. They will receive an email invitation — once accepted, they can view your repository
-

8. Workflow for project

Each time you update your project:

1. Make changes
 2. **Commit** (save snapshot locally) and **Push** (upload to GitHub)
-

Common Issues (Quick Fixes)

- **Git not installed**
 - Install Git from <https://git-scm.com>
 - **Authentication errors**
 - Re-login to GitHub in PyCharm
 - **Push rejected**
 - Click **Pull** first, then push again
-

Tips

- Commit often (after small changes)
- Use clear commit messages
- Keep repos **private** for assignments unless told otherwise
- Add .gitignore (PyCharm can do this automatically)



Gamer Accounts

You are tasked with building a Gamer Accounts System to manage player accounts.

Your program will read and manipulate account data stored in a file and provide useful functionality for administrators of an online gaming platform.

On Exam Day, you will add extra features.

You may not use dictionaries, or the CSV module. You may only use what you have learned in MTU.

Data Set Format

This will be stored as CSV.



ID: Player IDs start with PRO (Pro Gamers) and CAS (Casual Players).

Paid Membership Fees: "Yes" or "No"

Days: Integer, days since last password reset.

Status: "Active", "Locked", "Disabled".

A sample dataset might be

```
CAS10456,No,198,Active
PRO23456,Yes,22,Active
CAS11890,No,85,Locked
CAS10987,Yes,150,Active
CAS13456,Yes,10,Active
CAS12789,Yes,23,Active
PRO22777,No,110,Locked
CAS11223,Yes,15,Active
PRO20345,Yes,30,Active
PRO21888,Yes,60,Active
PRO21001,Yes,14,Disabled
CAS10234,Yes,45,Active
CAS13999,No,95,Disabled
PRO22334,Yes,20,Active
PRO21456,No,11,Active
CAS13012,No,60,Locked
PRO23012,Yes,45,Active
CAS12045,Yes,90,Active
PRO20789,Yes,5,Active
PRO23999,No,100,Active
```

Core Functionality – Starter Project

Your program will include the following menu of options, offered repeatedly as long as the user does not choose Quit:

Option 1: View all account data

Write a function that receives all 4 lists, and displays the accounts in a formatted table.

- Print  if Paid is 'Yes', otherwise use .
- Print 'Casual' after IDs starting with 'CAS'.
- Print 'Pro' after IDs starting with 'PRO'.
- Highlight those accounts with over 90 days since last password-reset by adding an alert  at the end of the row

Sample rows are:

CAS10234	Casual		Active
PR020789	Pro		Active
PR023999	Pro		Active 

Option 2: Delete a record

Write a function that receives all 4 lists.

In the function ask the user for the ID.

If the ID is not found, display an appropriate message.

If the ID is found, delete all data associated with the record along with an appropriate success message.

Option 3: Add a new record

Write a function that receives all 4 lists.

In the function prompt for an ID.

Print an appropriate error message if the ID already exists.

If the ID does not exist, add it and add default values for the other lists:

-  Paid = "No"
-  Days = 0
-  Status = "Active"

Print an appropriate success message.

Option 4: Update Status

Write a function that receives 2 lists: ID and status.

Ask the user for an ID.

Show a message if ID not found.

Otherwise ask the user for the new status and update that account. Print a success message.

Option 8 - Quit & Save

Write a function that receives all 4 lists.

Save the data from the lists back to the file

Program ends.

Starter code for the main

```
def menu():
    filename = "college.txt"
    ids, gdpr, days, status = read_file(filename)
    choice = ""
    while choice != "8":
        print("\nMenu")
        print("1. View all players")
        print("2. Delete a player")
        print("3. Register new player")
        print("4. Update player status")
        print("5. Placeholder for Exam")
        print("6. Placeholder for Exam")
        print("7. Placeholder for Exam")
        print("8. Quit and save")
        choice = input("Enter choice: ")

    if choice == "1":
        pass
    elif choice == "2":
        pass
    elif choice == "3":
        pass
    elif choice == "4":
        pass
    elif choice == "5":
        pass
    elif choice == "6":
        pass
    elif choice == "7":
        pass
    elif choice == "8":
        pass
        print("Data saved. Goodbye.")
    else:
        print("Invalid choice.")
```